

Assignment 1

Milestone 1

Part (a) - What was attempted

For the first milestone, we attempted the following techniques – MLP, Autoencoder, and Ensemble Learning. The batch sizes used for the three levels of noise were different (32, 128, 128 -> none, low, high) because of the total number of samples in the data. Same batch sizes were used in other methods as well.

1. MLP – We implemented a base architecture for a classification MLP in all three settings (eras, target_5_val, and target_10_val). The dimensions used for training were [input_size, 32, 64, 32, num_classes]. This model was trained for **50** epochs in all noise settings.
2. AutoEncoder – To obtain strong, noise-less features from the datasets, we employed an autoencoder architecture. The first part of the autoencoder employed a reconstruction paradigm, using MLP regression models with the MSE Loss, which was done for 100 epochs. After this, we replaced the decoder part of the MLP and trained a classification layer for 20 epochs. The dimensions of the autoencoder are [input_size, 32, 16, 8, 16, 32, input_size] and the classifier are [input_size, 32, 16, 8, num_classes].
3. Ensemble - This was our second noise mitigation technique wherein we trained five individual weak classifiers (each being an MLP model of different dimensions) for all three targets separately. Each individual classifier was trained for **15** epochs each. During testing, the final model predictions were made based on majority voting and used to calculate the accuracy of the model. For reference, the following were the dimensions for all 5 models -
 - a. Model 1: [input_size, 64, 32, num_classes]
 - b. Model 2: [input_size, 128, 64, 32, 16, num_classes]
 - c. Model 3: [input_size, 32, num_classes]
 - d. Model 4: [input_size, 64, 128, 64, 24, num_classes]
 - e. Model 5: [input_size, 32, 64, 128, num_classes]

Part (b) - Tabulation of results (Noise Level vs Technique Used)

Noise Level	Base MLP	Autoencoder	Ensemble
None	89.06	19.91	84.10
Low	78.43	26.48	78.30
High	54.31	18.80	54.69

Table 1. Accuracy for target class: 'eras'.

Noise Level	Base MLP	Autoencoder	Ensemble
None	95.90	85.38	96.58
Low	94.35	83.63	94.35
High	86.61	78.69	86.48

Table 2. Accuracy for target class: 'target_5_val'.

Noise Level	Base MLP	Autoencoder	Ensemble
None	97.78	84.87	97.61
Low	95.40	77.82	95.43
High	87.74	75.68	88.51

Table 3. Accuracy for target class: 'target_10_val'.

Part (c) - Conclusions

We see that in most cases, MLP is a good enough (if not better) classifier suited to our task.

Autoencoder model – In the case of the AE model, we observe a stark drop in the results. We hypothesize the following reasons:

1. The number of training samples to capture the dependencies in the data is not enough.
2. The discrete nature of the data is not used in the reconstruction of features of the autoencoder. Since most of the columns are discrete, we should employ a CE loss instead of a MSE loss (current paradigm) to reconstruct features. This may significantly show improvement in the results.

Ensemble model – The ensemble model showed comparable results to the MLP, beating it in several cases as well. The reasons for such results are as follows:

1. The ensemble model effectively eliminates similar noise's impact by the virtue of using multiple randomly initialized models to its advantage, with different architectures as well.
2. The fact that we could not train the ensemble models for 50 epochs each was due to time/resource constraints. If we could have done that, we believe the results would be better for the ensemble models.

Milestone 2

Part (a) - What was attempted

For the second milestone, we implemented the TabPFN model (Transformer) and as part of the Sequence Models we implemented LSTM. We combined our LSTM with the ensemble noise mitigation technique and compared the results. The batch sizes used for the three levels of noise were different (16, 128, 128 -> none, low, high) because of the total number of samples in the data. Same batch sizes were used in all methods.

1. TabPFN – Although this was a pretrained model it required certain adjustments. Firstly, in the case of the label ‘eras’, we were dealing with 12 classes but the model could only handle 10 classes. To counter this, we used an ensemble of one vs all classifiers and finally made predictions based on majority voting. Secondly, the model could only train on 1024 samples at once. Thus, we need to cleverly sample our training samples in batches of size 1024 while also making sure that model sees at least 2 classes in each batch. However, this posed another problem. This method was leading to a large number of models being trained and saved and the system ran out of memory before being able to see all samples. To deal with this, we devised the following ‘bagging’ approach - build a TabPFN classifier, fit on a training batch, make predictions on the entire test set at once using this classifier, save the predictions and delete the classifier instance itself. During inference, to calculate the final class for a sample in the test set, we used the majority voting concept to make our final prediction. Although this method deals with most of the issues, it still takes a long time to make predictions for large datasets (datasets with high and low noises, in our case, each having nearly 300,000 samples).
2. LSTM - The model is fairly simple. We made use of the LSTM architecture that comes with pytorch and set parameters and hyperparameters as - hidden_size = 64, num_layers = 4, learning_rate = 0.0005, weight_decay = 0.001. We also modified the dataset to now enable the input to be a sequence of len 10 and ensured that all samples in a sequence should belong to the same era.
3. LSTM Ensemble - We combined our sequence model LSTM with the preferred noise mitigation technique that we got in Milestone 1. In this, we train 3 weak individual LSTMs and make our final predictions using majority voting. Each LSTM is trained for 30 epochs.

Part (b) - Tabulation of results (Noise Level vs Technique Used)

Noise Level	MLP	TabPFN	LSTM	LSTM Ensemble
None	89.06	90.39	09.19	7.34
Low	78.43	70.47	94.57	96.89
High	54.31	43.55	87.71	93.05

Table 1. Accuracy for target class: 'eras'.

Noise Level	MLP	TabPFN	LSTM	LSTM Ensemble
None	95.90	92.82	24.14	25.38
Low	94.35	74.31	78.70	81.41
High	86.61	56.56	61.07	63.20

Table 2. Accuracy for target class: 'target_5_val'.

Noise Level	MLP	TabPFN	LSTM	LSTM Ensemble
None	97.78	92.76	27.66	23.11
Low	95.40	77.90	83.87	85.73
High	87.74	60.89	57.43	66.13

Table 3. Accuracy for target class: 'target_10_val'.

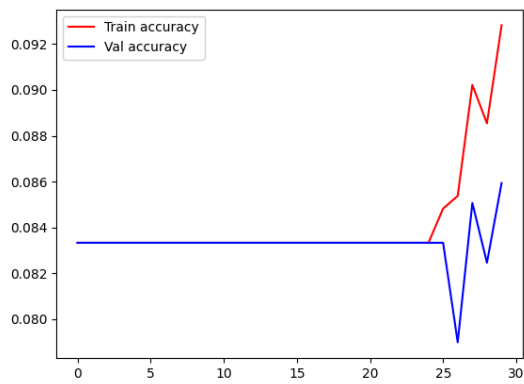
Part (c) - Conclusions

TabPFN - We see that TabPFN performs quite well when dealing with no noise data and is able to get stable prediction metrics for all different labels. With noise data, the predictions are not bad, but they are inferior as compared to the base MLP model. This is possibly due to the small amount the model learns when it only sees 1024 samples at a time. Given this small size to fit each small model, of which we take a max voting, the model cannot learn intricate representations that are present in the noisy samples in the data. Hence, the approach needs to increase the batch size from 1024 (which is not possible due to computational restrictions).

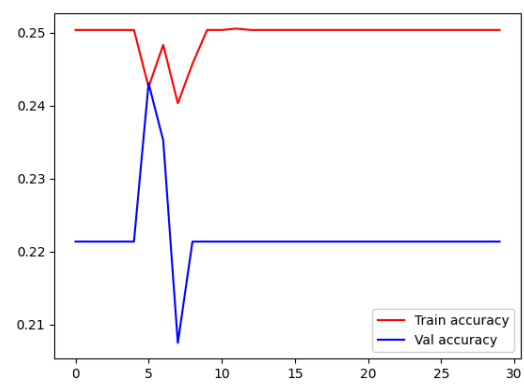
LSTM - We see that LSTMs perform quite badly for no noise dataset. However, this is not because of the dataset entries, rather it is due to the small size of the dataset. With just 7800 training samples the model is not able to learn anything and can't make conclusive decisions. This is supported by the fact that even though the noisy dataset has varying levels of noise, the LSTM beats MLP quite significantly for 'eras' classification task. For target_5_val and target_10_val, the LSTM does struggle compared to MLP but with larger training time, the bridge can be shortened.

LSTM ensemble - LSTM itself was performing well, and continued to perform even better with just three models as part of the ensemble. This shows sequence modeling + noise mitigation technique is a good combination to deal with the classification tasks for the given data.

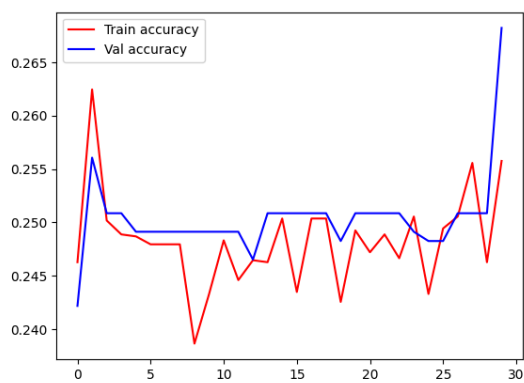
Part (d) - Supporting Visualisations



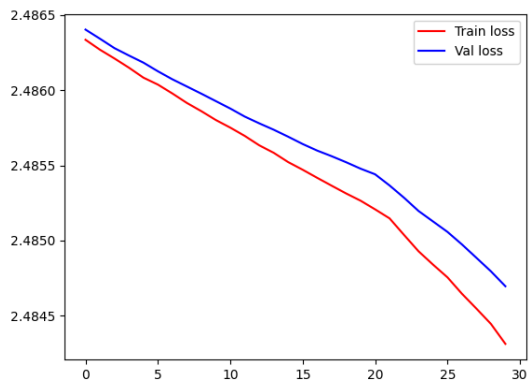
Accuracy - no noise - era



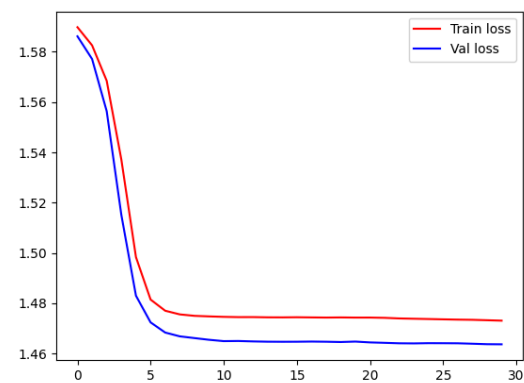
Accuracy - no noise - target_5_val



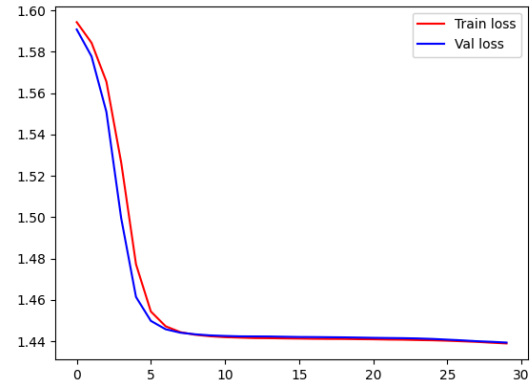
Accuracy - no noise - target_10_val



Loss - no noise - era



Loss - no noise - target_5_val



Loss - no noise - target_10_val

We should theoretically perform such experiments with a higher learning rate ($1e-4$), since it seems like these graphs are trapped due to minimal movement in the hyperspace. Upon trying this experiment, we see better results. However, for the experiments with noise, this learning rate seems to be too high, and the model fails to converge as fast as with a smaller learning rate.

The results for the re-run experiments are:

Target Class	MLP	LSTM	LSTM Ensemble
Eras	89.06	42.61	76.20
Target_5_val	95.90	73.53	85.05
Target_10_val	97.78	72.93	65.63

Table 1. Accuracy for no noise data – re-run experiments